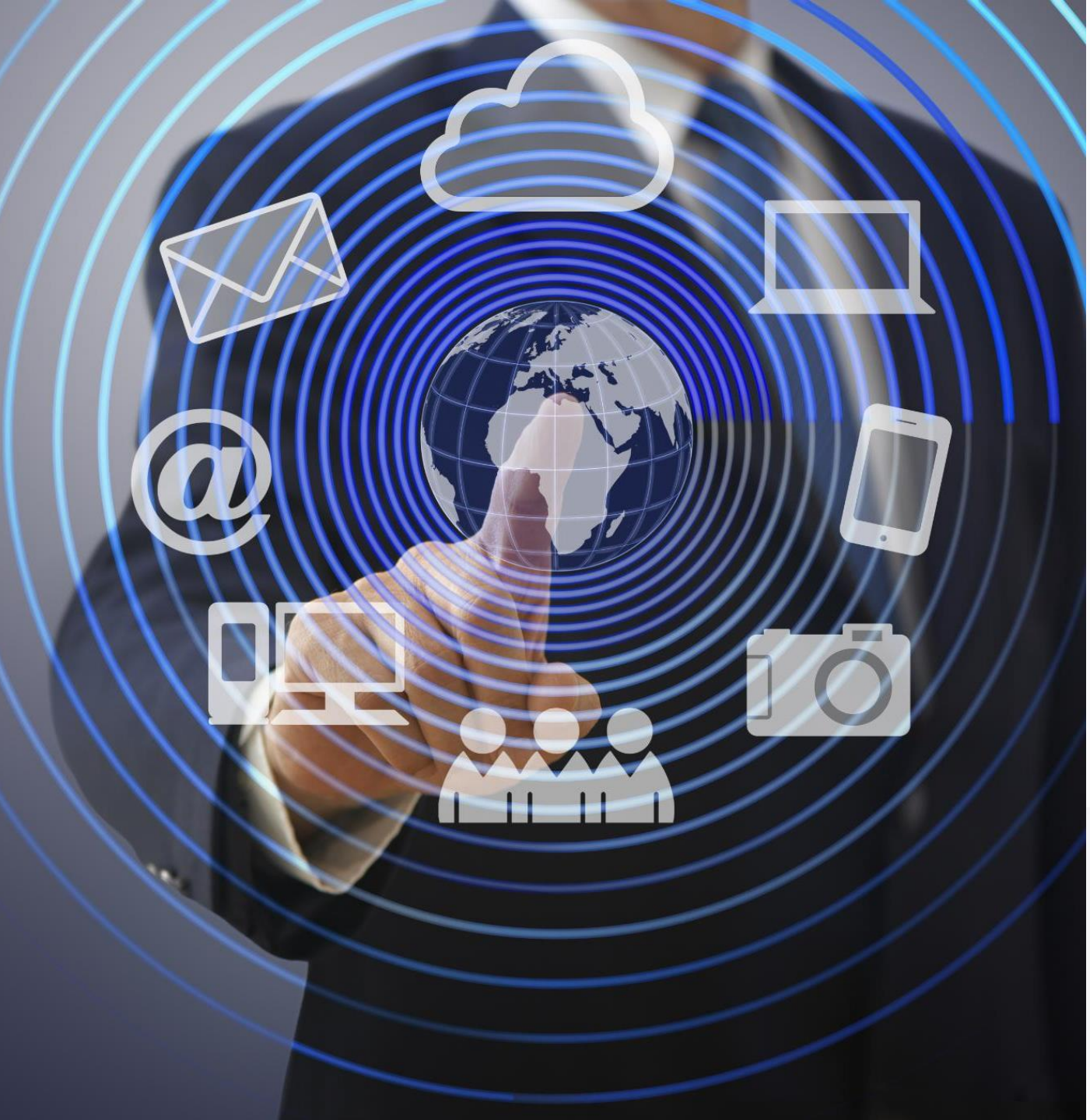
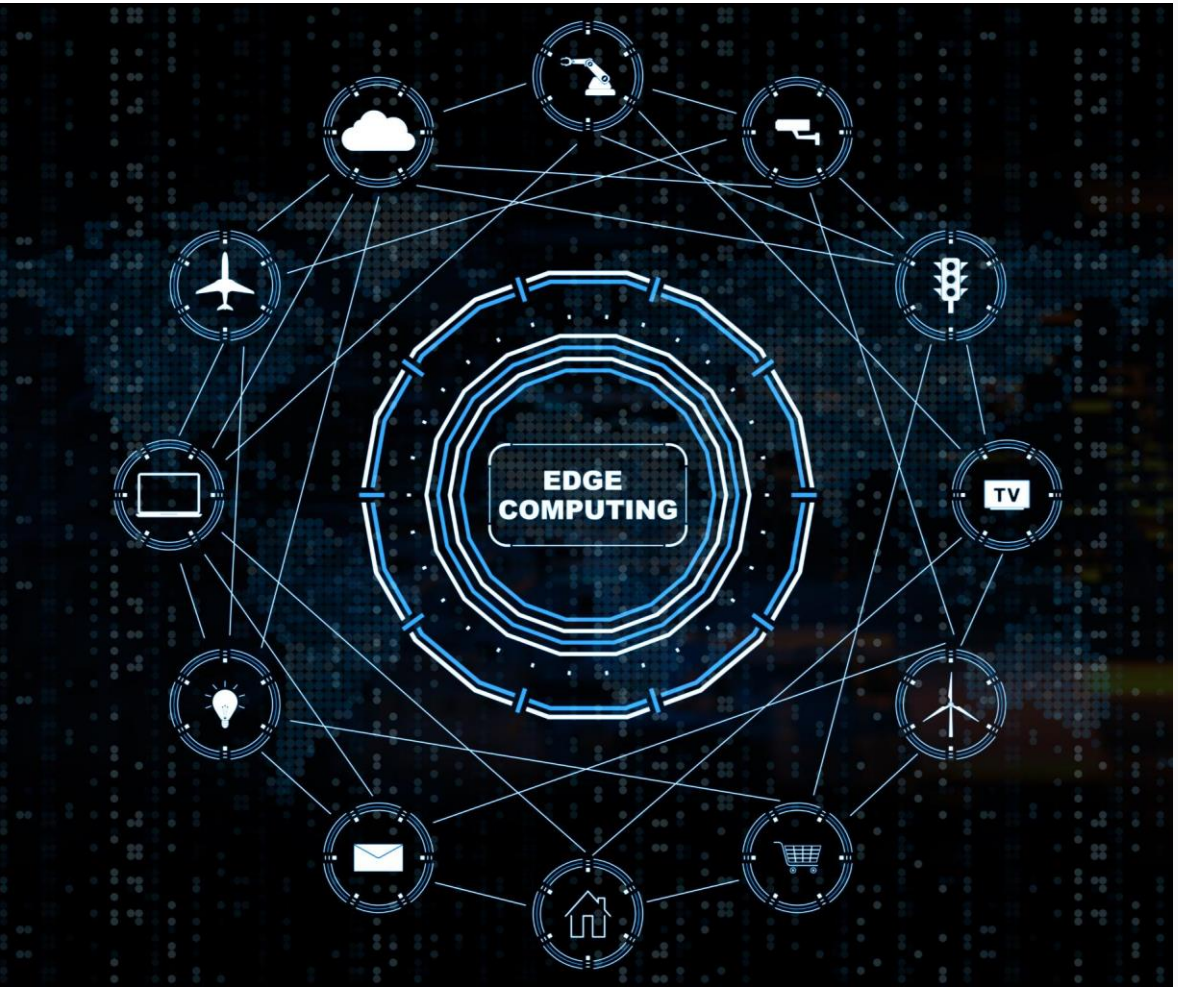


Unified Cost Governance for AI Agents Across Copilot Studio, Azure AI Foundry, and Microsoft Agent Framework

Integrating cost management for AI platforms effectively

Framing the Cost Governance Challenge

Cost Governance Overview: One Operating Model, Three Agent Platforms



Challenges of Multi-Platform Governance

Multiple agent platforms cause fragmented reporting, inconsistent tagging, and difficulty attributing costs accurately.

Unified Cost Governance Model

A single model aligns stakeholders and normalizes diverse platform units into consistent cost events for better control.

Cost Multipliers and Controls

AI agents add unique cost factors like tokens and tool calls, requiring design-time policies and runtime quotas for control.

Practical Operational Example

Without shared identifiers, costs split across systems cannot be reconciled, complicating cost governance efforts.

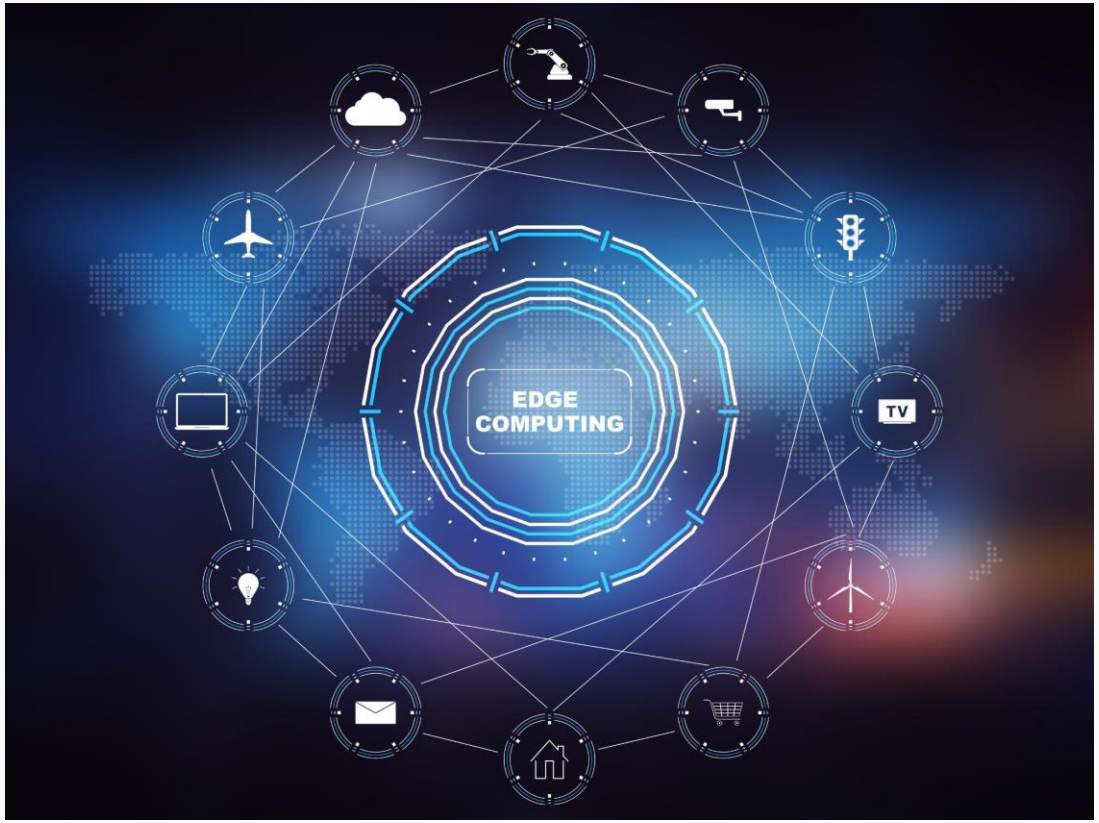
Objectives: Transparency, Control, and Finance-Ready Allocation

- Stakeholder Objectives
  - Define what good means for engineering, product owners, and finance with tailored cost governance goals.
- Consistent Allocation
  - Map every agent to an owner, environment, and cost center to enable operational cost control.
- Measurement and Control
  - Collect usage telemetry and apply layered control policies for effective cost governance.
- Transparent Reporting
  - Integrate billing signals into accessible dashboards for clear financial reporting and optimization plans.

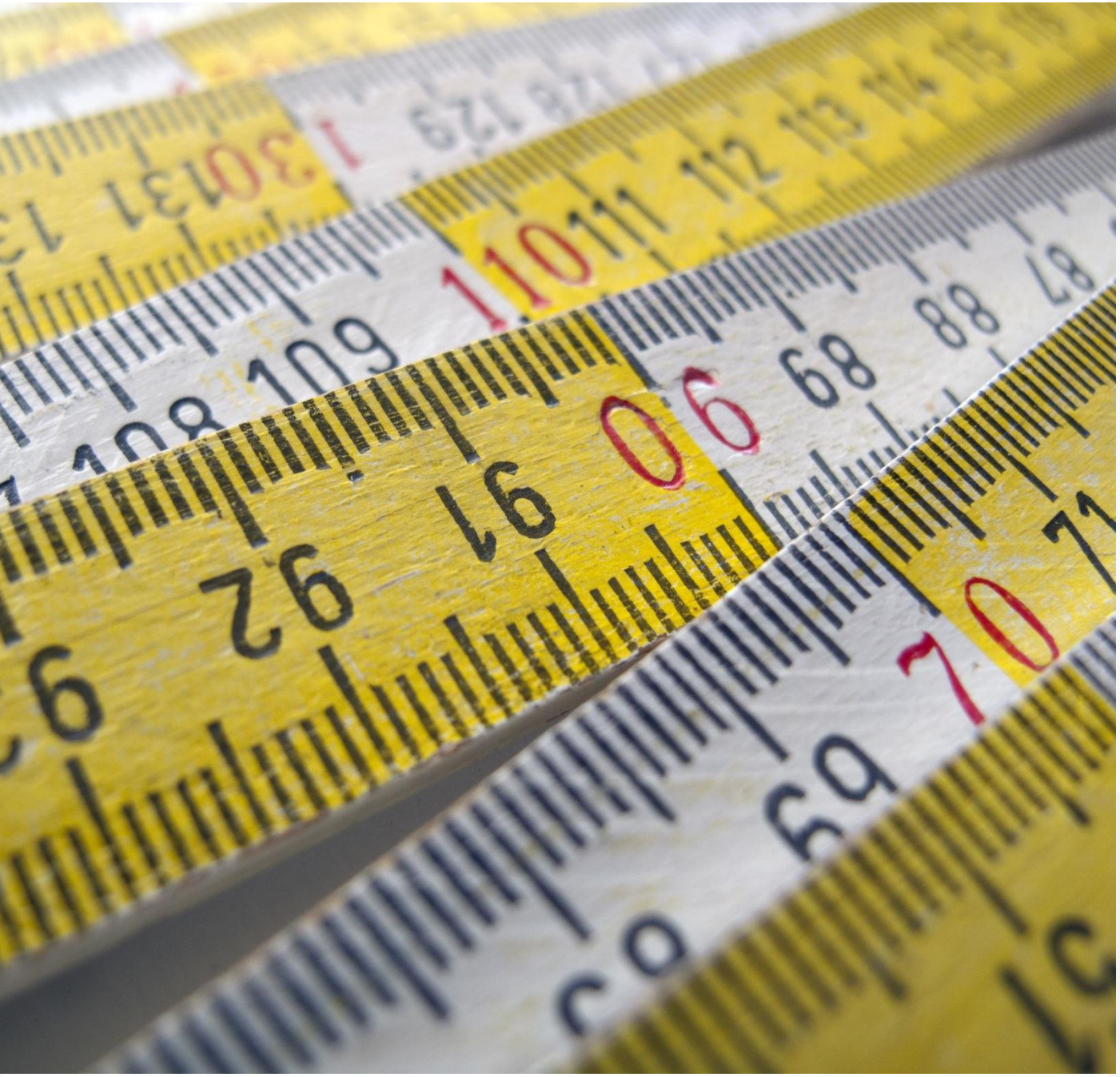
Why AI Agents Change the Cost Equation Compared to Traditional Apps

- Complexity of AI Agent Costs
  - AI agent costs depend on probabilistic compute and variable workflows, unlike fixed infrastructure costs in traditional apps.
- Key Cost Multipliers
  - Grounding, multiple actions, and autonomous triggers increase token usage and backend costs in AI agents.
- Importance of Cost Governance
  - Cost governance requires observability of model steps, tool calls, and runtime patterns to optimize spend and scale.
- Optimization Strategies
  - Tiered model usage and capacity strategies reduce costs while maintaining quality and preventing resource waste.

Scope and Assumptions: What We Govern Across the Fleet



- Agent Fleet Composition
  - Governance covers Copilot Studio, Azure AI Foundry, and custom agents with distinct cost and usage models.
- Governance Cross-Cutting Concerns
  - Four governance concerns include allocation, measurement, control, and reporting for comprehensive cost management.
- Allocation and Measurement
  - Agents must be registered with owners and generate consistent CostEvents with key telemetry and correlation IDs.
- Control and Reporting Mechanisms
  - Control includes model tiering and limits; reporting enables finance dashboards and operational analytics.



Measurement Foundations and Normalization

Unified Cost Governance Principles: Allocate, Measure, Control, Optimize, Report

Allocate: Accountability and Metadata

Tie every agent to an accountable owner, cost center, and environment as mandatory metadata for operational cost decisions.

Measure: Standardized Telemetry

Collect per-request usage data including model, tools used, and consumption to identify cost drivers and reconcile credits.

Control: Layered Guardrails

Apply preventive, runtime, and detect controls to limit spending and prevent cost overruns through policies and alerts.

Optimize and Report

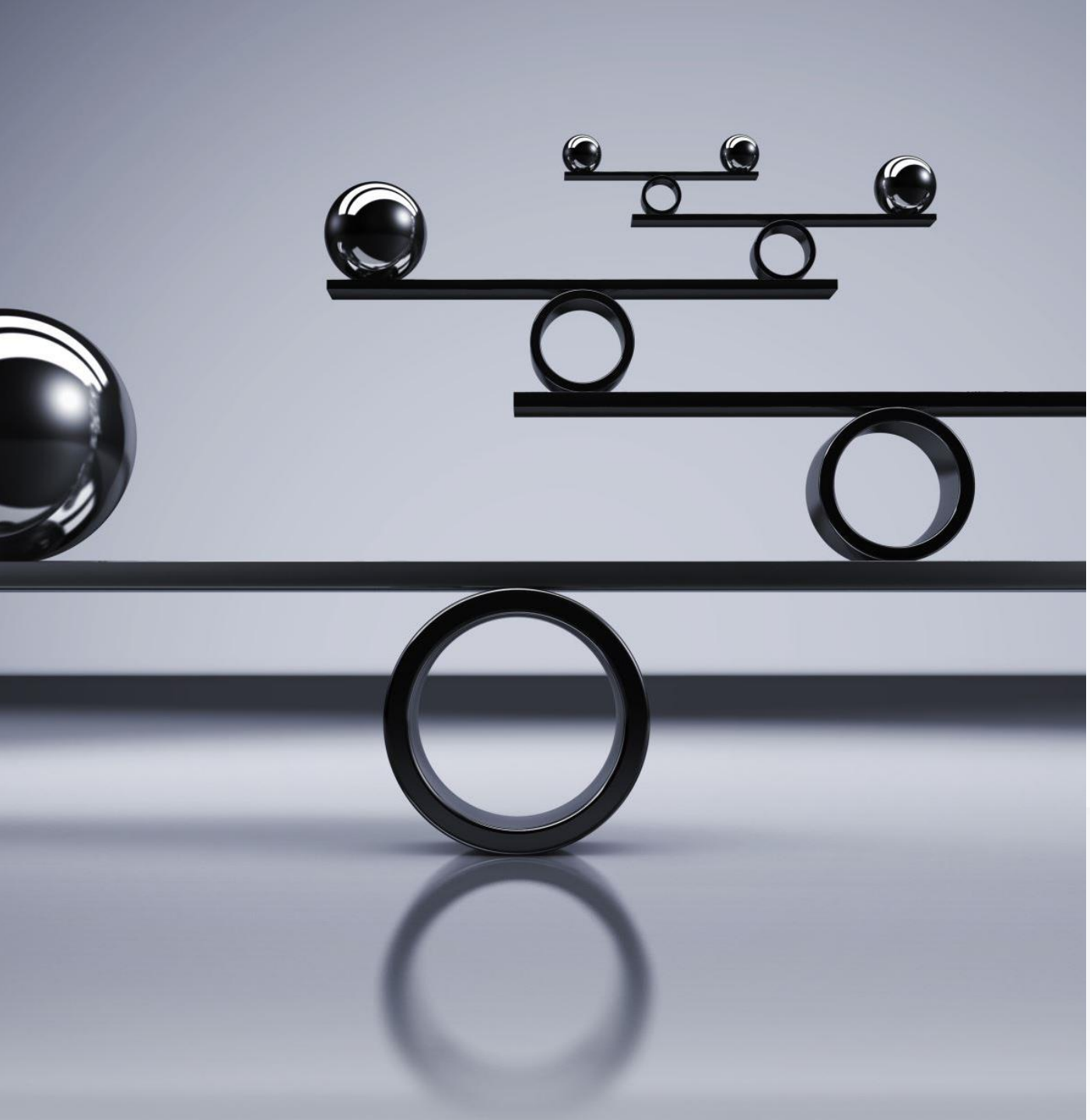
Continuously improve cost performance with optimizations and deliver role-based dashboards for operational and financial insights.

CostEvent Schema: The Minimal Record Needed for Showback and Optimization

FIELD	EXAMPLE	WHY IT MATTERS
platform	CopilotStudio | Foundry | AgentFramework	Normalizes source systems for unified reporting
agent_id / agent_name	HR-Policy-Agent / “HR Policy Helper”	Primary key for showback/chargeback and inventory
environment	dev | test | prod	Budget segmentation and control thresholds
owner / cost_center	HR Ops / CC-1427	Accountability and allocation to business units
channel	Teams | Web | API	Attribution by experience and adoption patterns
model	gpt-4o, o1-mini	Model choice drives cost, latency, quality and quotas
meter_unit	credits | tokens | PTU-hours	Supports normalization and cross-platform comparisons
quantity	2200 tokens / 18 credits	Raw usage used for thresholds and anomalies
estimated_cost	$0.012	Finance-ready rollups and budget tracking
correlation_id	trace-9e2f...	Reconciles gateway, runtime, and tool-call telemetry

Native Cost Signals by Platform: Mapping Credits, Tokens, and PTU-hours

PLATFORM	NATIVE UNIT	NATIVE VISIBILITY	UNIFICATION APPROACH
Copilot Studio	Copilot Credits	Agent analytics + billing/consumption management	Export/ingest credit consumption; enforce tenant budgets and allocations
Azure AI Foundry	Tokens + Azure meters; optional PTU	Foundry dashboards + Application Insights observability	Metered gateway emits token metrics; correlate with billing exports; track PTU utilization
Microsoft Agent Framework	Depends on backend (often tokens) + infra	App Insights/OpenTelemetry instrumentation + infra metrics	Route all model/tool calls via gateway; standardize headers/claims and CostEvent emissions

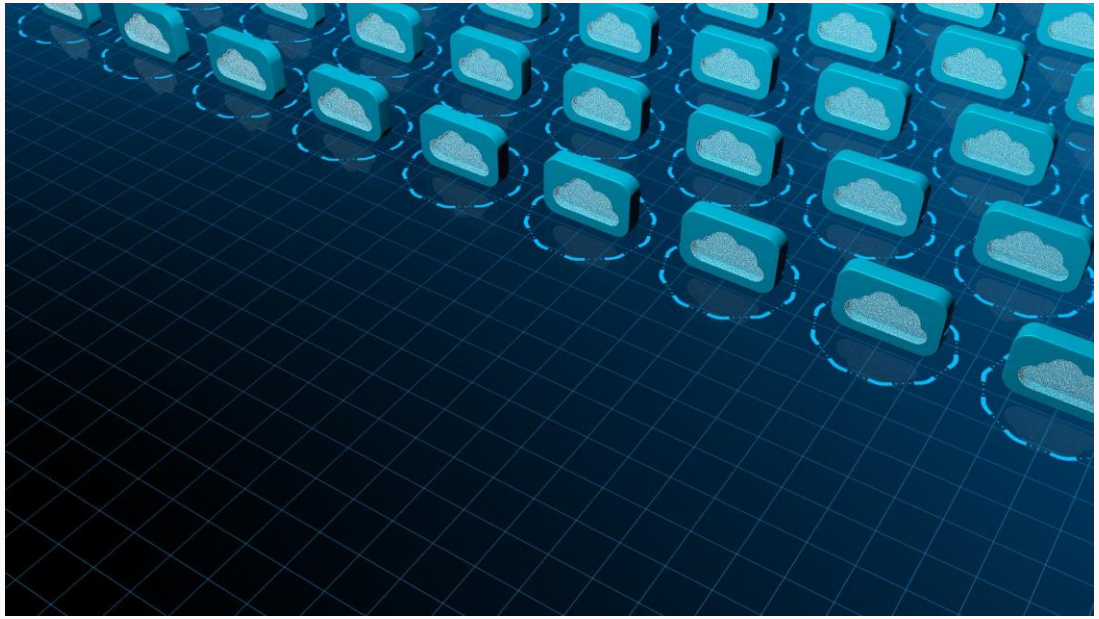


Platform-Specific Governance Levers

Copilot Studio Cost Governance: Credits, Billing Policies, and Forecasting

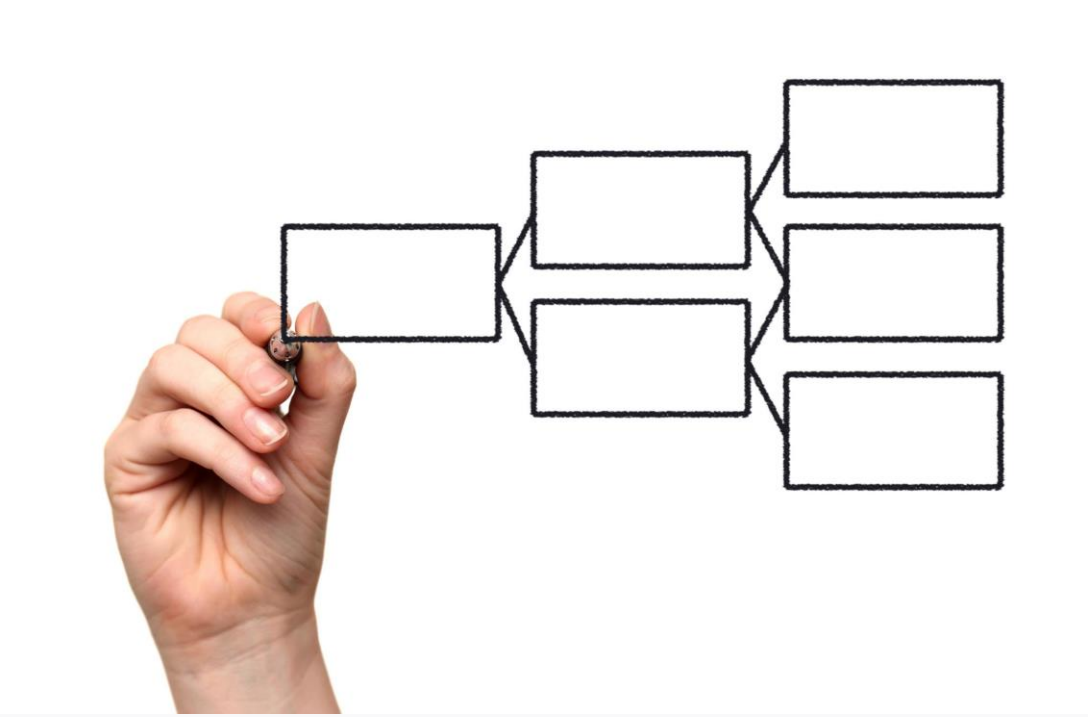
- Credits as Cost Units
  - Copilot Credits measure work units and are used as budgets, allocations, and constraints to manage costs effectively.
- Billing and Cost Design Review
  - Review credit-consuming features and estimate monthly usage before production deployment to control expenses strategically.
- Forecasting and Cost Estimation
  - Use agent consumption estimators to forecast credit usage and communicate costs to finance and management teams.
- Governance Controls and Allocation
  - Enforce ownership, environment separation, and channel-aware controls to manage tenant-level pooled credits and traffic spikes.

Azure AI Foundry Cost Governance: Tokens, PTU Strategy, and Observability



- Dual Consumption Models
  - Cost governance is based on pay-as-you-go tokens and provisioned throughput units for predictable capacity management.
- Quota and Rate Limits
  - Token quotas and rate limits distribute usage fairly across consumers to prevent resource starvation in shared deployments.
- Observability and Metrics
  - Integrating Application Insights collects usage, latency, and error metrics to correlate cost spikes with operational issues.
- Model Selection Strategy
  - Governance enforces choosing optimized models for routine tasks and premium models for high-value scenarios.

Microsoft Agent Framework Governance: Consistent Metering for Custom Agents



- Architectural Requirements
  - Standardized context propagation and centralized metering ensure consistent cost governance for custom agents.
- Centralized Metering Benefits
  - A centralized AI gateway stamps metadata, enforces quotas, and emits token metrics for unified billing.
- Orchestration Telemetry Insights
  - Telemetry tracks agent steps, tool usage, retries, and controls orchestration patterns to manage costs.
- Governance and Observability
  - Governance defines agent usage patterns and limits, with OpenTelemetry aiding cost-event traceability.

Architecture, Controls, and Reporting

Metered AI Gateway Pattern: APIM + Application Insights for Token-Level Telemetry

- Centralized Identity and Allocation
  - The gateway adds consistent metadata like agent ID and cost center to requests for accurate telemetry grouping.
- Rate Limiting and Quota Management
  - Token quotas and rate limits prevent any one agent from consuming all available tokens, ensuring fair resource allocation.
- Token Metering and Cost Estimation
  - The gateway extracts token usage data to emit custom metrics and estimate costs in Application Insights.
- Trace Correlation and Diagnostics
  - Trace identifiers are propagated to join gateway logs with agent runtime traces for comprehensive diagnostics.

Control Framework: Prevent, Limit, Detect (Design-Time, Runtime, and Operations)

- Prevent Controls at Design-Time
  - Prevent controls enforce policies during design and promotion, including model selection and feature governance.
- Limit Controls at Runtime
  - Limit controls act during runtime to stop overspending using quotas, rate limits, and budget alerts.
- Detect Controls in Operations
  - Detect controls monitor operations with dashboards, alerts, and runbooks to enable cost visibility and response.
- Policy-as-Code and Optimization
  - Integrating controls as code and incorporating FinOps reviews ensures ongoing cost optimization and predictability.

Dashboards and Reporting: Role-Based Views for Engineering, Product, and Finance

Engineering Dashboards

Engineering dashboards provide near-real-time diagnostics including token usage, latency, error rates, and root-cause analysis tools.

Product and Business Insights

Product dashboards connect spend to adoption and outcomes, tracking KPIs such as cost per conversation and channel mix.

Finance and FinOps Reporting

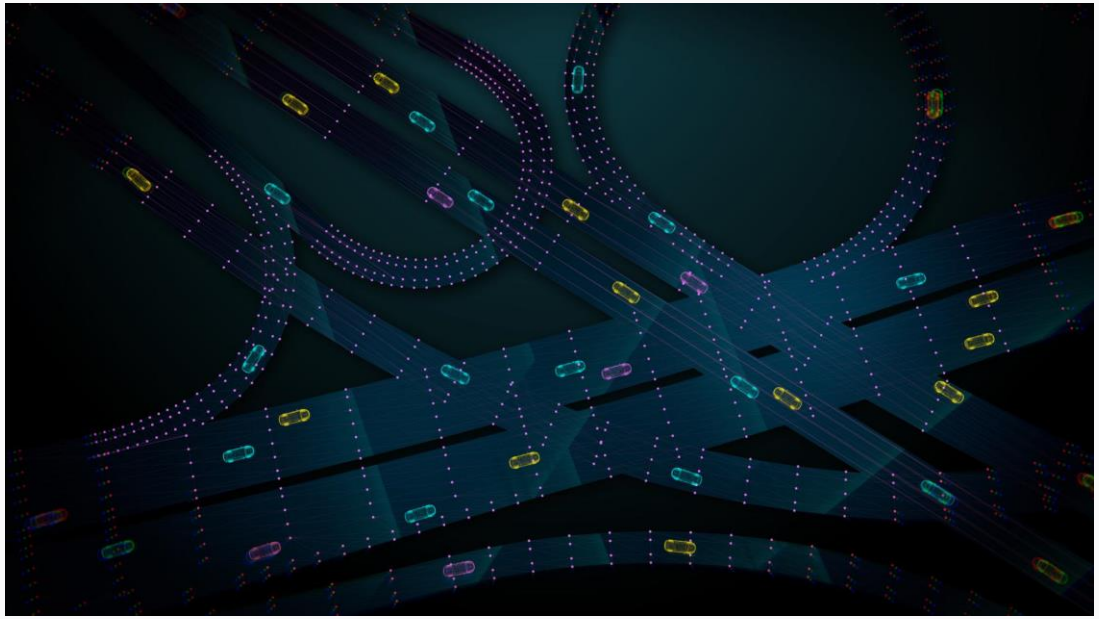
Finance reports focus on budget allocation, cost forecasting, and anomaly detection for better predictability and control.

Unified Reporting Architecture

Integrating telemetry, cost exports, and inventory into a single Power BI model supports comprehensive, customizable reports.

Execution Roadmap and Operationalization

Implementation Roadmap: From Inventory to Optimization Loop

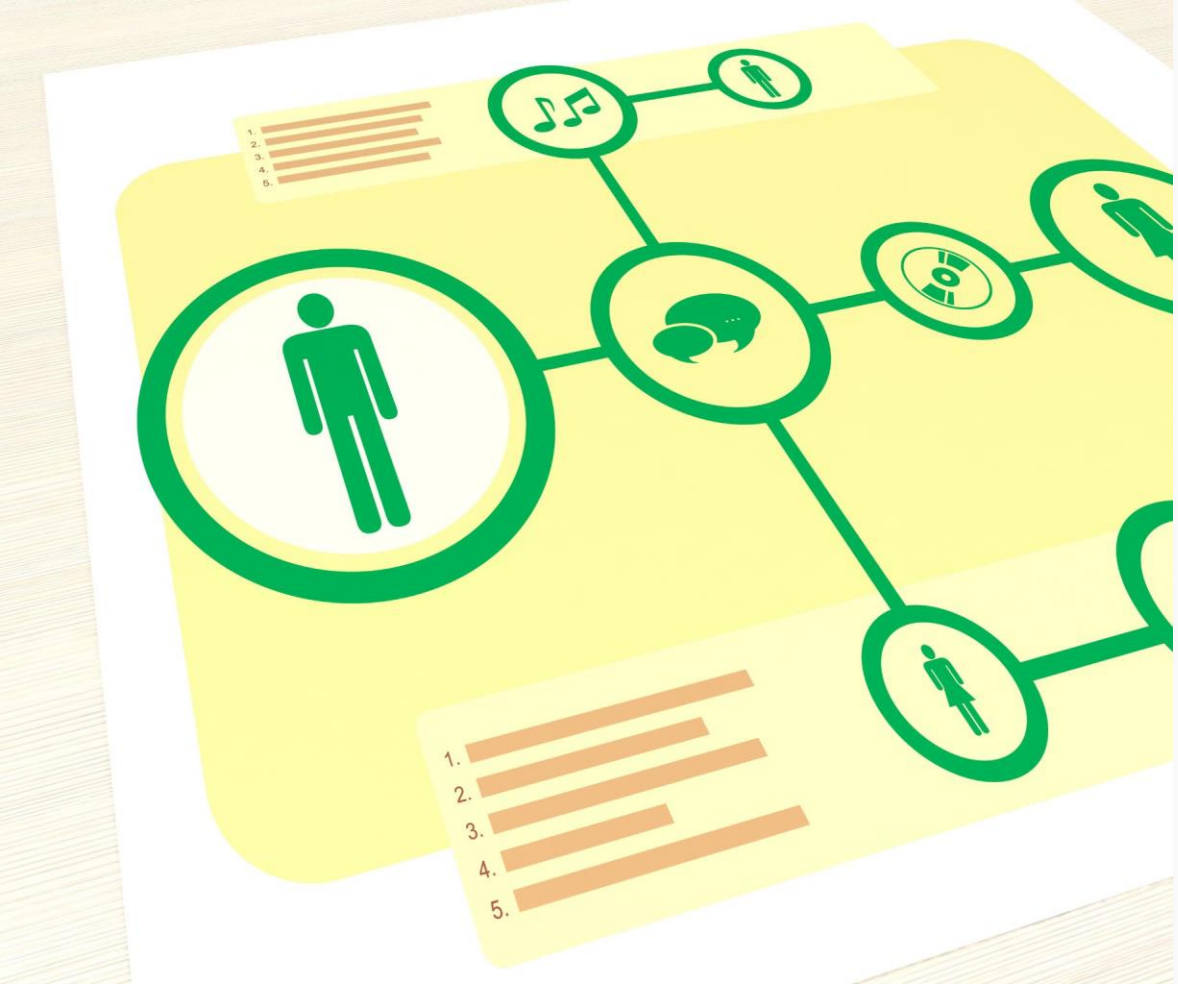


- Phase 1: Inventory and Allocation
  - Create an agent registry with mandatory fields and enforce naming and tagging policies for consistent reporting.
- Phase 2 & 3: Telemetry and Metering
  - Implement telemetry with correlation strategies and deploy metered gateways to track token usage and costs.
- Phase 4: Governance Extension
  - Extend billing and monitoring to Copilot Studio, integrating credit consumption into unified reports.
- Phase 5: Optimization Loop
  - Use regular FinOps reviews to optimize costs by refining model tiers, caching, and reservation sizing.

Common Risks and Mitigations: Runaway Spend, Noisy Neighbors, and Attribution Gaps

- Runaway Loops Risk
  - Recursive calls and retries can rapidly increase token usage causing runaway costs in autonomous agents.
- Noisy Neighbor Problem
  - One application consuming shared model quota leads to throttling and increased retries for others.
- Cost Attribution Gaps
  - Inconsistent tagging prevents proper cost allocation, making optimization difficult and guesswork.
- Governance and Capacity Risks
  - Over-provisioning and governance friction slow innovation; automation and monitoring improve control.

Reference Guidance and Source Materials to Anchor the Program



Authoritative Guidance Importance

Anchoring cost governance in authoritative guidance aligns stakeholders and accelerates adoption with repeatable patterns.

Billing and Consumption Monitoring

Utilize billing rates, Copilot Credits management, and usage estimators to forecast and monitor agent consumption effectively.

Enterprise-wide Governance Controls

Employ unified cost visibility, cost center tagging, and real-time alerts to prevent cost overruns organization-wide.

Reporting and Analytics Integration

Leverage Microsoft Cost Management exports with Power BI or Fabric to create finance-ready dashboards and advanced analytics.

Next Steps: Choose KPIs, Enforce Metadata, Deploy Metering, and Establish FinOps Cadence

Selecting and Measuring KPIs

Choose primary KPIs like cost per agent monthly and make them measurable for effective cost governance.

Enforcing Metadata Compliance

Mandate ownership and environment labels for agents with automated governance through templates and CI checks.

Deploying Metering Infrastructure

Route model calls through gateways to enforce quotas and emit token usage metrics for cost attribution.

Establishing FinOps Cadence

Set monthly reviews to monitor spenders, diagnose cost drivers, and assign optimization with measurable targets.